Computational Neuroscience

Chapter (2)
Deep Neural Networks

Edited by

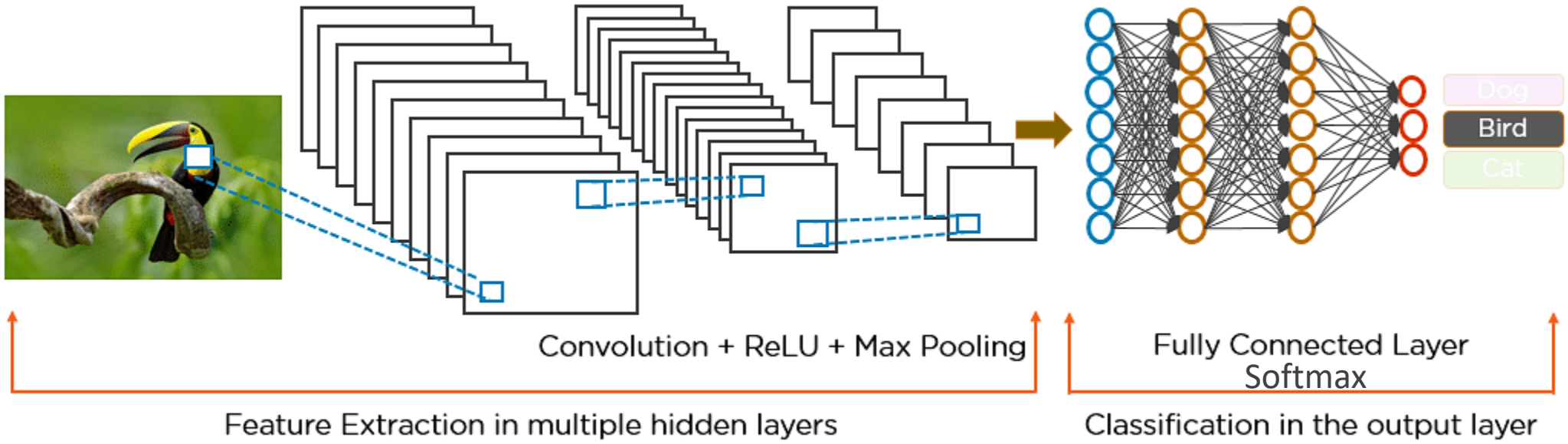
A. Prof. Noha El-Attar

General Types of ANN

- Perceptron
- Feed Forward Neural Network
- Multilayer Perceptron
- Convolutional Neural Network
- Radial Basis Functional Neural Network
- Recurrent Neural Network
- LSTM – Long Short-Term Memory
- Sequence to Sequence Models
- Modular Neural Network
- Generative Adversarial Networks (GAN)

Deep Neural Network

- Deep Neural Network uses **composite of simple functions** (e.g., **ReLU, sigmoid, tanh, softmax)** to create complex non-linear functions.



Machine Learning vs Deep Learning

- **Machine Learning** is AI algorithms capable of automatic adaptation with minimal human interference, such as **linear regression, random forest, decision tree, ANN**, etc..
- **Deep Learning** is a subset of Machine Learning using **neural networks** to mimic the learning process of the human brain.

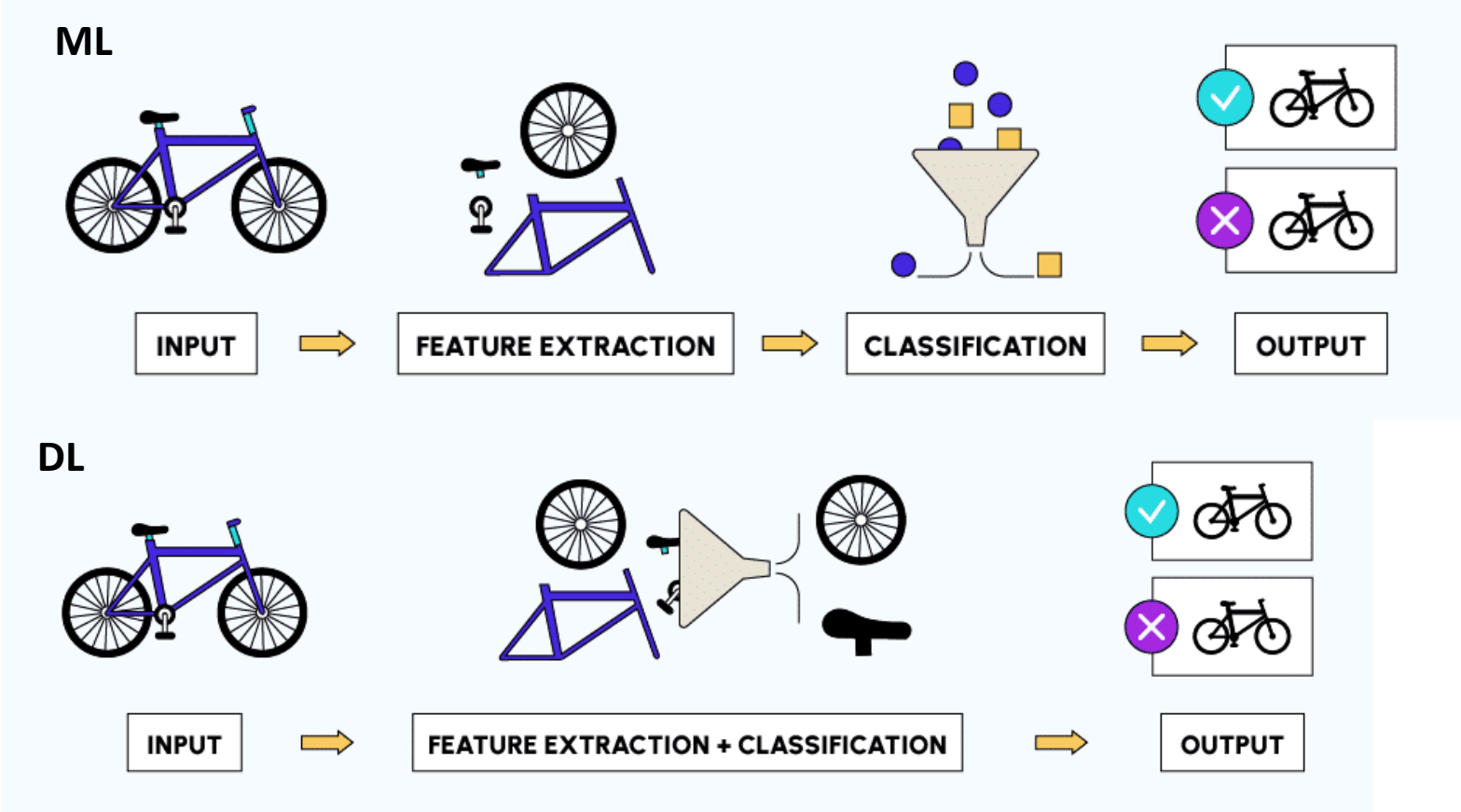
- **Machine Learning** enables **training on smaller datasets**, but requires more human intervention to learn and correct mistakes.
- **Deep Learning** requires **larger volumes of training data**, but learns from its own environment and mistakes.

Machine Learning vs. Deep Learning:
Feature Engineering

- Feature engineering is a key step in the model building process. It is a two-step process:
  - **Feature extraction**
  - **Feature selection**

Feature Extraction

In feature extraction, we extract all the required features for our problem statement and transform raw data into numerical features that can be processed while preserving the information in the original data set



Feature Selection

- The method of reducing the input variable to your model by using only relevant data and getting rid of noise in data.

What happened if you train a deep learning model on a small data set?

Underfitting: **Poor performance on the training data and poor generalization to other data**
Overfitting: **Good performance on the training data, poor generalization to other data**
Both resulting in poor performance.

What should be the appropriate size of training data while training a deep learning model?

- Approximately, you need at least 5000 training examples in order to get "acceptable" results. And, you need at least 10 million training examples to get near-human performance

ML vs DL

Aspect	Machine Learning	Deep Learning
Data Dependency	Typically requires structured data.	Well-suited for unstructured data, e.g., images, audio, text.
Feature Engineering	Requires manual feature engineering.	Automatically learns features from data.
Network Depth	Shallower network architectures.	Involves deep neural networks with multiple hidden layers.
Training Data Size	Works well with smaller datasets.	Benefits from large datasets for better performance.
Interpretability	Easier to interpret and explain results.	Complex models can be less interpretable.
Computation Resources	Requires fewer computational resources.	Demands significant computational power (GPUs/TPUs).
Use Cases	Common for tasks like regression, classification.	Ideal for tasks like image recognition, NLP, and more complex problems.
Training Time	Faster training with smaller models.	Longer training due to deeper architectures.

Deep Learning Algorithms

- Convolutional Neural Networks (CNNs)
- Long Short Term Memory Networks (LSTMs)
- Recurrent Neural Networks (RNNs)
- Generative Adversarial Networks (GANs)
- Radial Basis Function Networks (RBFNs)
- Multilayer Perceptrons (MLPs)
- Self Organizing Maps (SOMs)
- Deep Belief Networks (DBNs)
- Restricted Boltzmann Machines(RBMs)
- Autoencoders

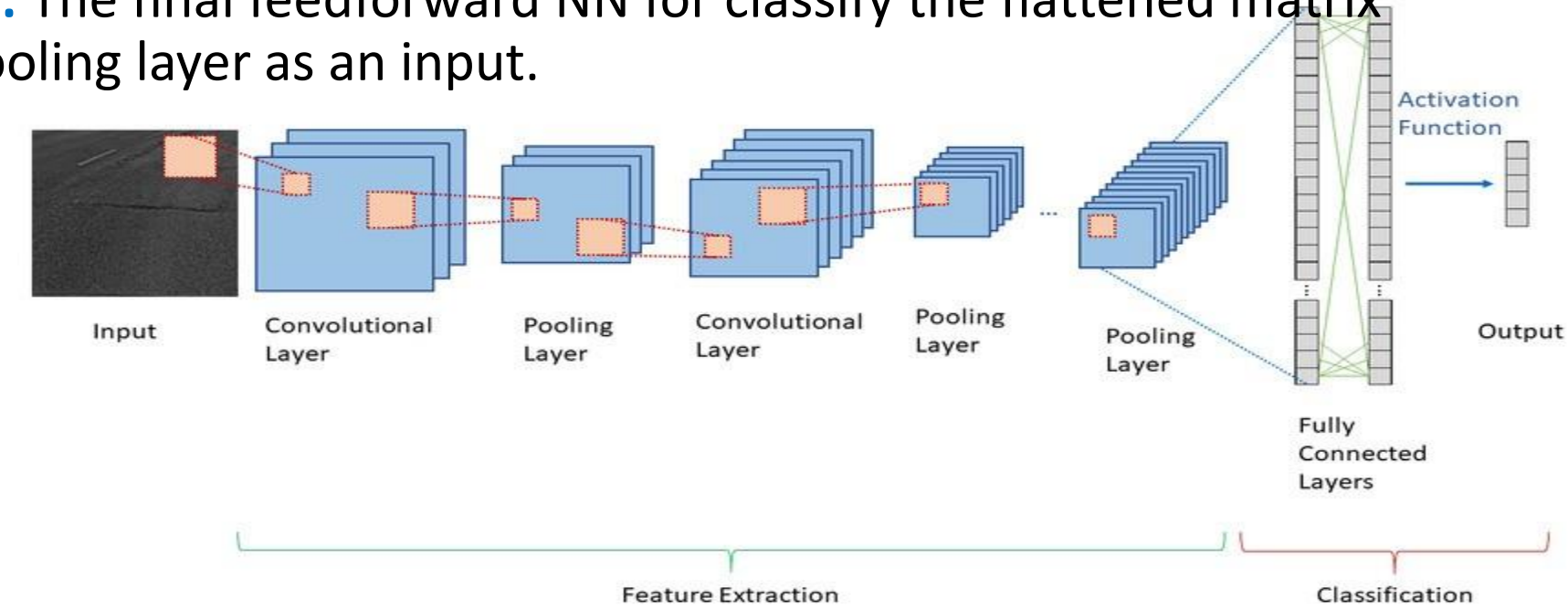
Convolutional Neural Network (CNN)

- The idea of this type of algorithms is to enable machines to view the world as humans do, perceive it in a similar manner, and even use the knowledge for a multitude of tasks such as **Image & Video recognition, Image Analysis & Classification, Media Recreation, Recommendation Systems, Natural Language Processing**, etc.
- The advancements in Computer Vision with Deep Learning have been constructed and perfected with time, primarily over one particular algorithm — a **Convolutional Neural Network**.

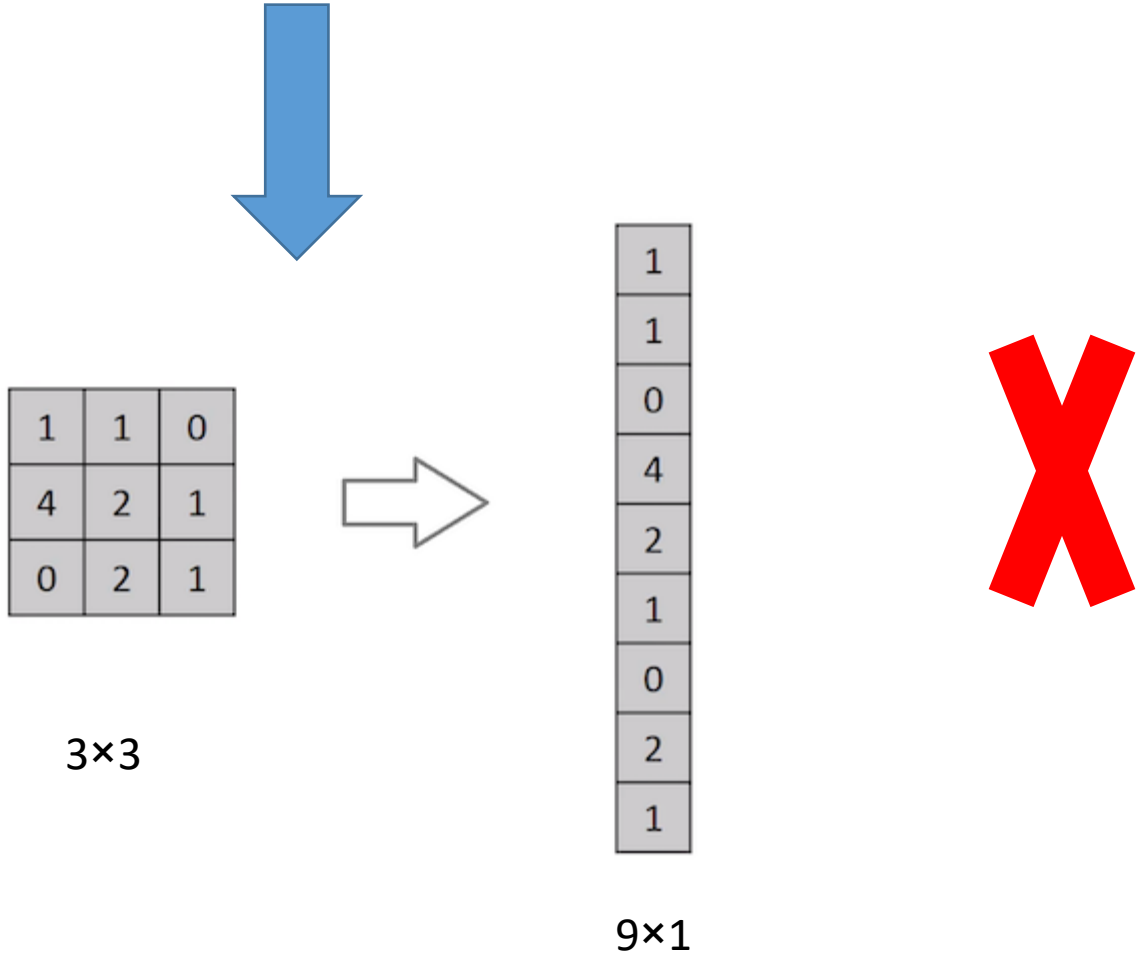
Convolutional Neural Network (CNN)- cont..

CNN consists of multiple layers:

- **Convolution Layer:** that has several filters to perform the convolution operation.
- **Pooling Layer:** a down-sampling operation that reduces the dimensions of the feature map, then converts the resulting two-dimensional arrays from the pooled feature map into a single, long, continuous, linear vector by flattening it.
- **Fully Connected Layer:** The final feedforward NN for classify the flattened matrix that come from the pooling layer as an input.

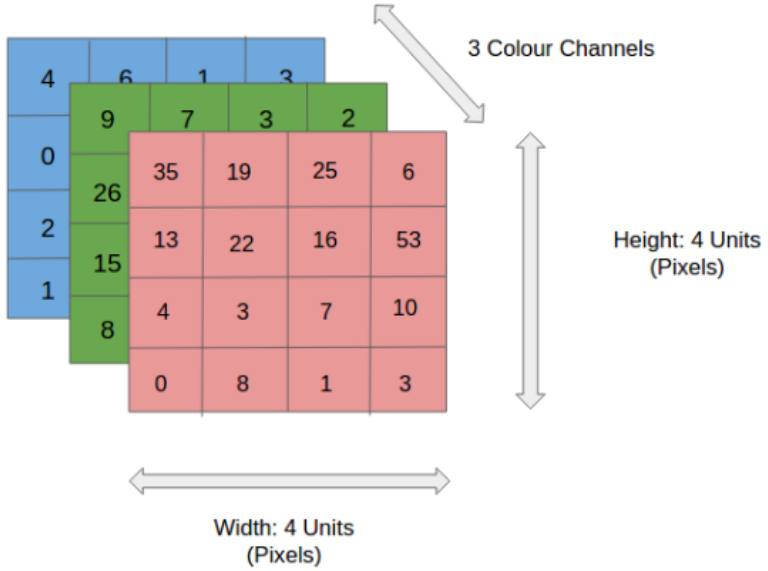


Can we process the image like this?



- **Images** are read as pixels and it is expressed as matrix **(NxNx3)** — (width, height, and channel).

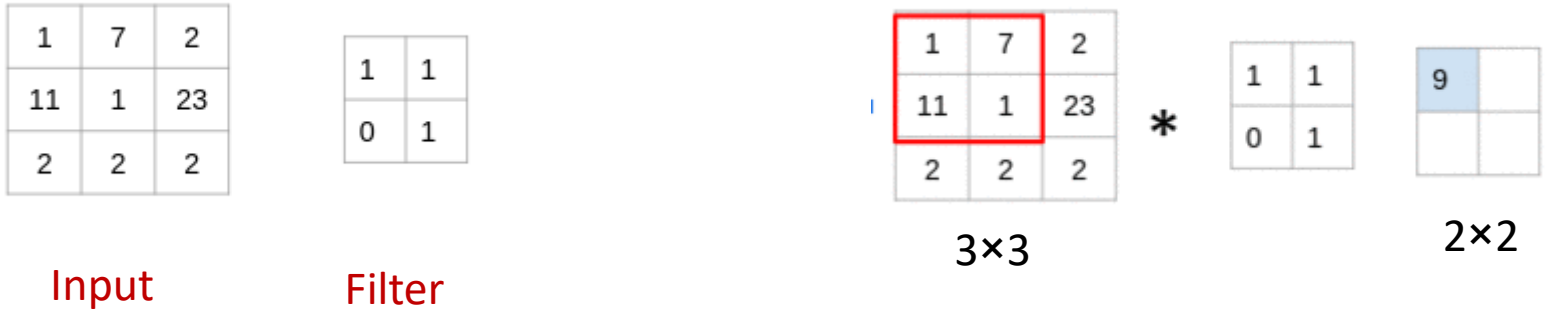
4x4x3 RGB Image



- What about an image with size **8K (7680×4320)**? Big number of pixels.
- The **role of ConvNet is to reduce the images** into a form that is easier to process, **without losing features** that are critical for getting a good prediction.
- This is important when we are to design an architecture that is not only good at learning features but also scalable to massive datasets.

1. Convolution layer:

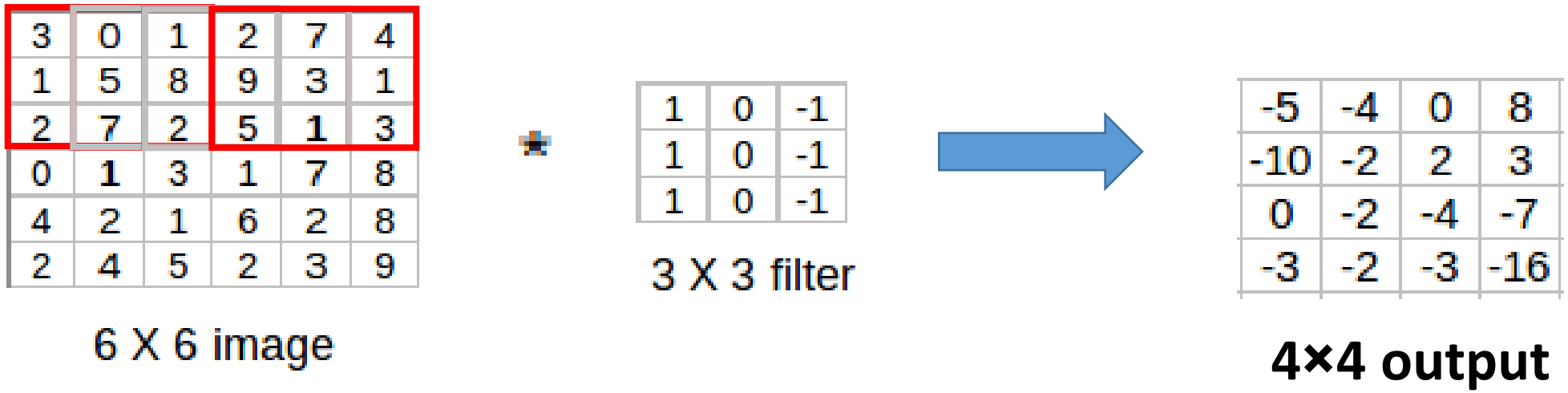
The Convolutional Layer makes use of a set of **learnable filters (kernels)**.
A filter, or kernel, is a small matrix of weights that slides over the input data, performs element-wise multiplication with the part of the input it is currently on, and then sums up all the results into a single output pixel.



(1x1 + 7x1 + 11x0 + 1x1) = 9
(7x1 + 2x1 + 1x0 + 23x1) = 32
(11x1 + 1x1 + 2x0 + 2x1) = 14
(1x1 + 23x1 + 2x0 + 2x1) = 26

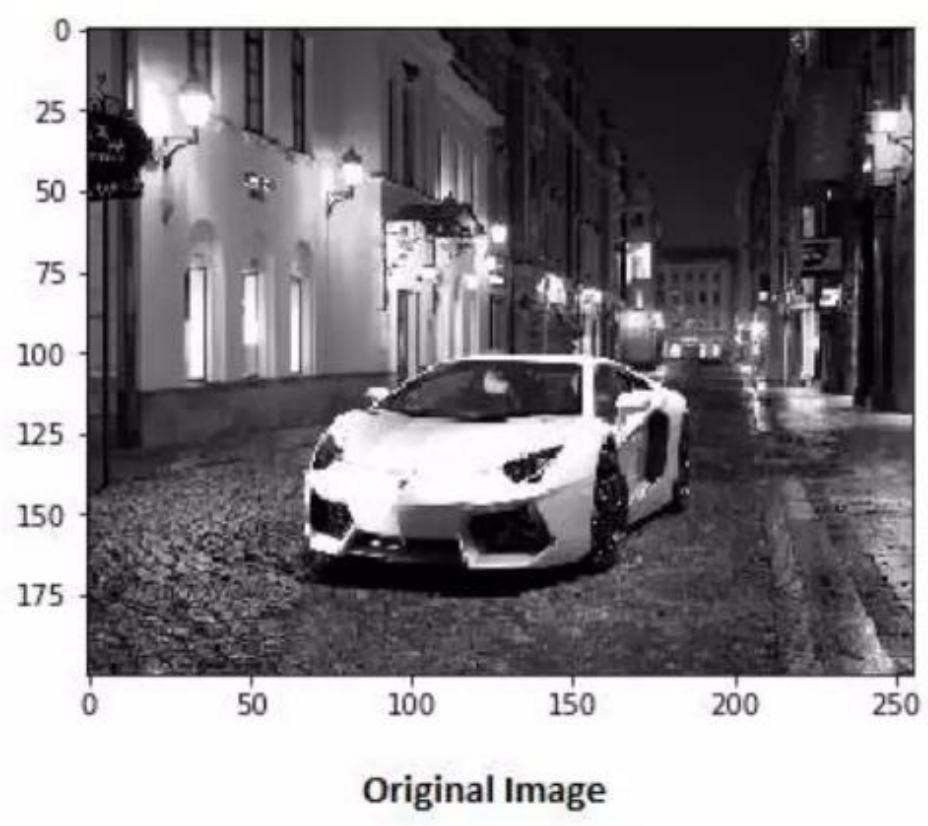
Example (2)

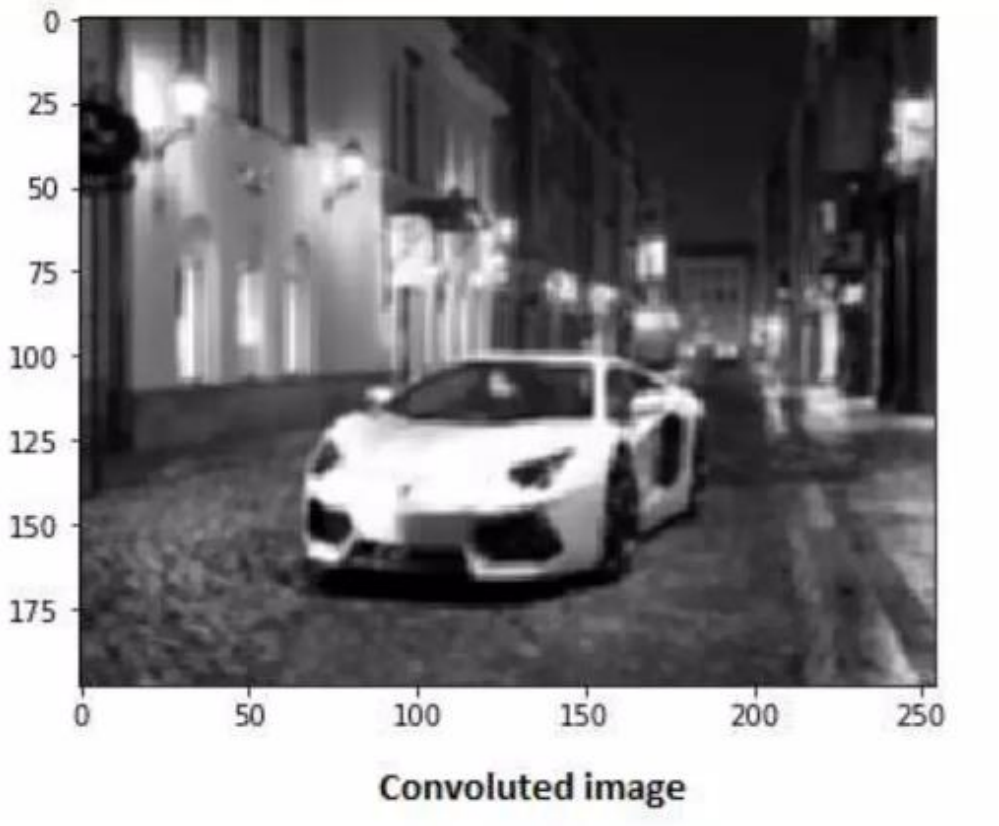
- If you have 6 X 6 grayscale image, and you will apply 3x3 filter, what is the size of the output convoluted image.



- **Is there a rule to compute the output?**
- if the input is n X n and the filter size is f X f, then the output size will be **(n-f+1) X (n-f+1)**

Difference between:

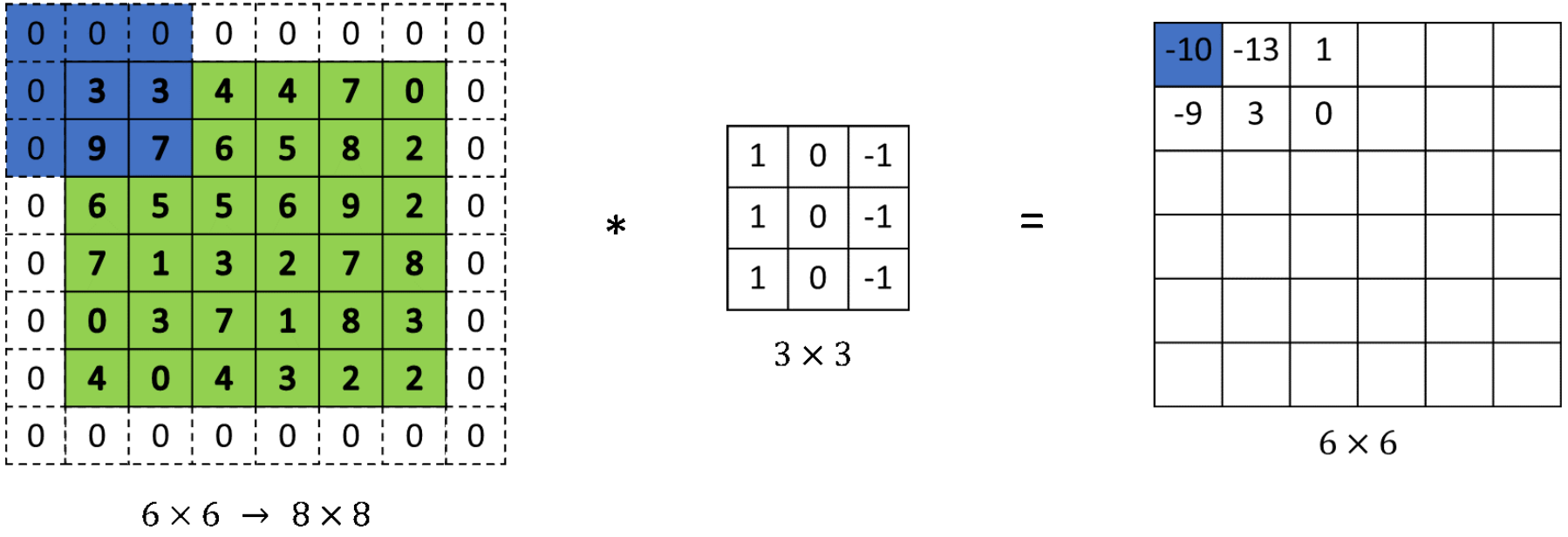




There are two main disadvantages in the traditional convolution process:

- Every time we apply a convolutional operation, **the size of the image shrinks**
- **Pixels present in the corner of the image are used only a few number** of times during convolution as compared to the central pixels. Hence, we do not focus too much on the corners since that can **lead to information loss**.
- **To overcome these issues**, we can **pad the image with an additional border**, (we add one pixel all around the edges with **zero** value).

This means that the input will be an 8 X 8 matrix (instead of a 6 X 6 matrix). Applying convolution of 3 X 3 on it will result in a 6 X 6 matrix which is the original shape of the image.



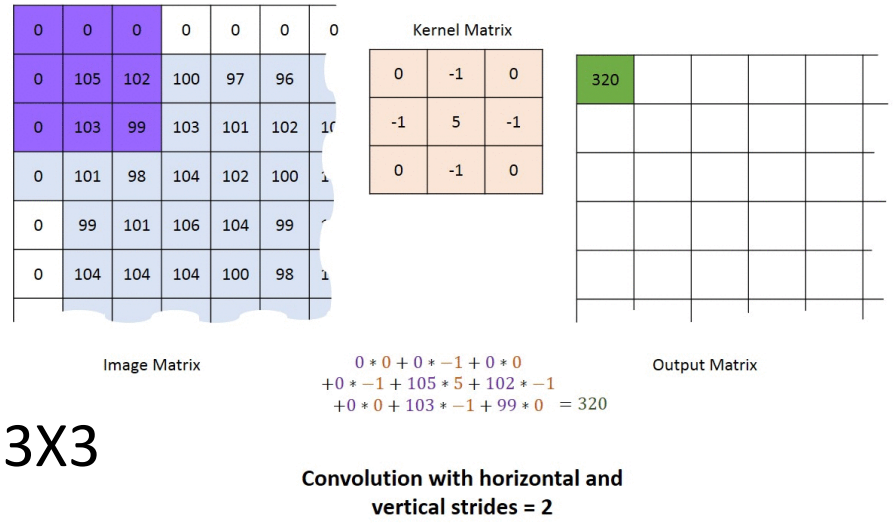
Input: n X n
Padding: p
Filter size: f X f
Output: (n+2p-f+1) X (n+2p-f+1)

How can we choose padding value?

- **Valid:** It means no padding. If we are using valid padding, the output will be (n-f+1) X (n-f+1)
- **Same:** Here, we apply padding so that the output size is the same as the input size,
- n+2p-f+1 = n
- p = (f-1)/2

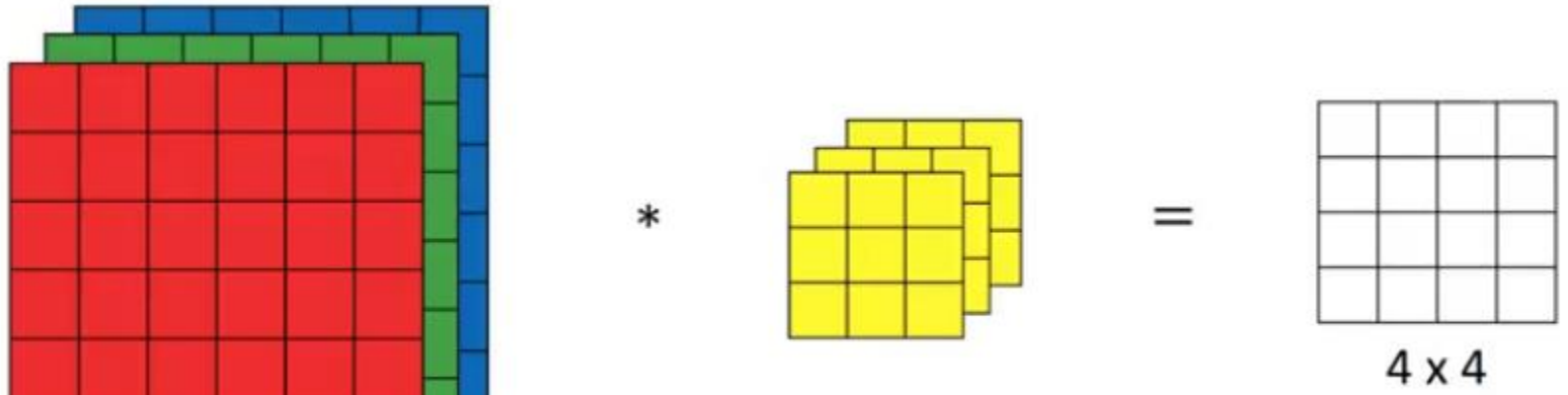
Strided Convolutions

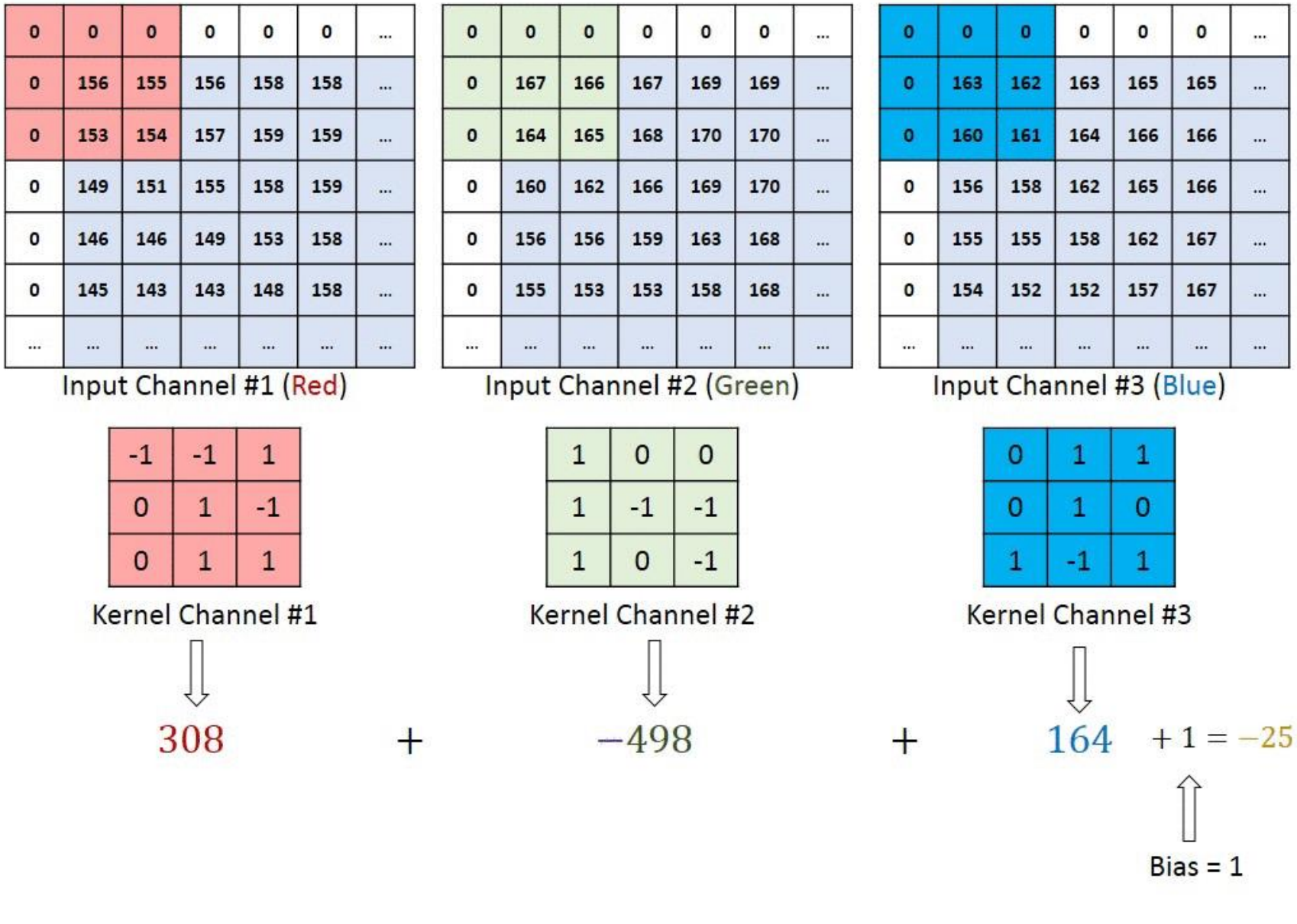
- Suppose we choose a stride of 2. So, while convoluting through the image, we will take two steps – both in the horizontal and vertical directions separately. The dimensions for stride *s* will be:
- **Input:** n X n 5X5
- **Padding:** p 1
- **Stride:** s 2
- **Filter size:** f X f 3X3
- **Output:** [(n+2p-f)/s+1] X [(n+2p-f)/s+1] = 3X3



Convolutions Over Volume (RGB)

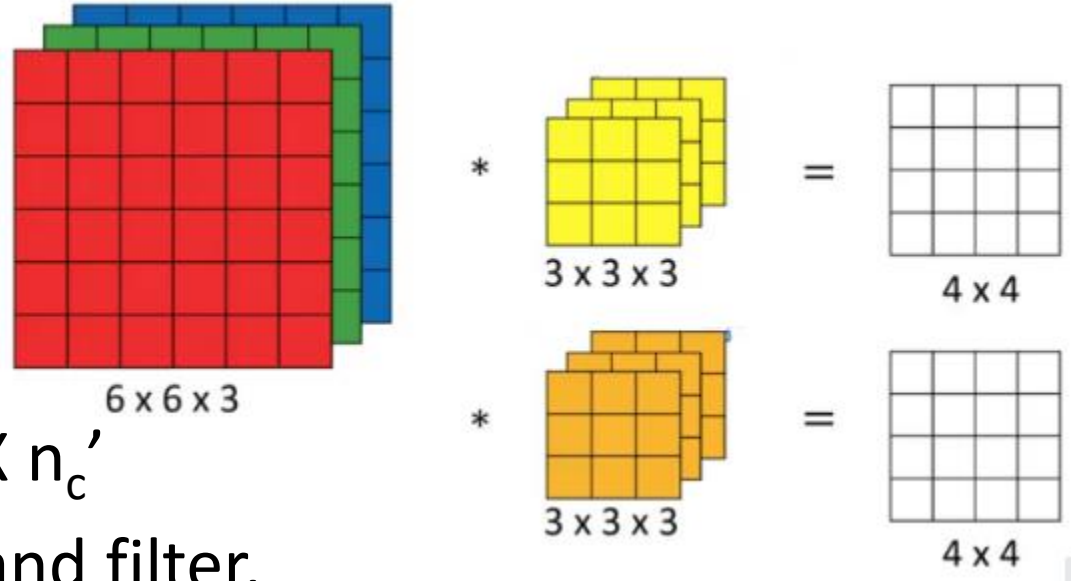
- **Input:** 6 X 6 X 3
- **Filter:** 3 X 3 X 3
- The dimensions above represent the height, width and channels in the input and filter.
- ***Keep in mind that the number of channels in the input and filter should be same.***
- This will result in an output of 4 X 4




0 0 0 0 0 0 ...
0 156 155 156 158 158 ...
0 153 154 157 159 159 ...
0 149 151 155 158 159 ...
0 146 146 149 153 158 ...
0 145 143 143 148 158 ...
... ...
Input Channel #1 (Red)
0 0 0 0 0 0 ...
0 167 166 167 169 169 ...
0 164 165 168 170 170 ...
0 160 162 166 169 170 ...
0 156 156 159 163 168 ...
0 155 153 153 158 168 ...
... ...
Input Channel #2 (Green)
0 0 0 0 0 0 ...
0 163 162 163 165 165 ...
0 160 161 164 166 166 ...
0 156 158 162 165 166 ...
0 155 155 158 162 167 ...
0 154 152 152 157 167 ...
... ...
Input Channel #3 (Blue)
-1 -1 1
0 1 -1
0 1 1
Kernel Channel #1
1 0 0
1 -1 -1
1 0 -1
Kernel Channel #2
0 1 1
0 1 0
1 -1 1
Kernel Channel #3
308
+
−498
+
164
+ 1 = −25
Bias = 1
Output
-25 ... ...
... ...
... ...
... ...
... ...

Multiple filters

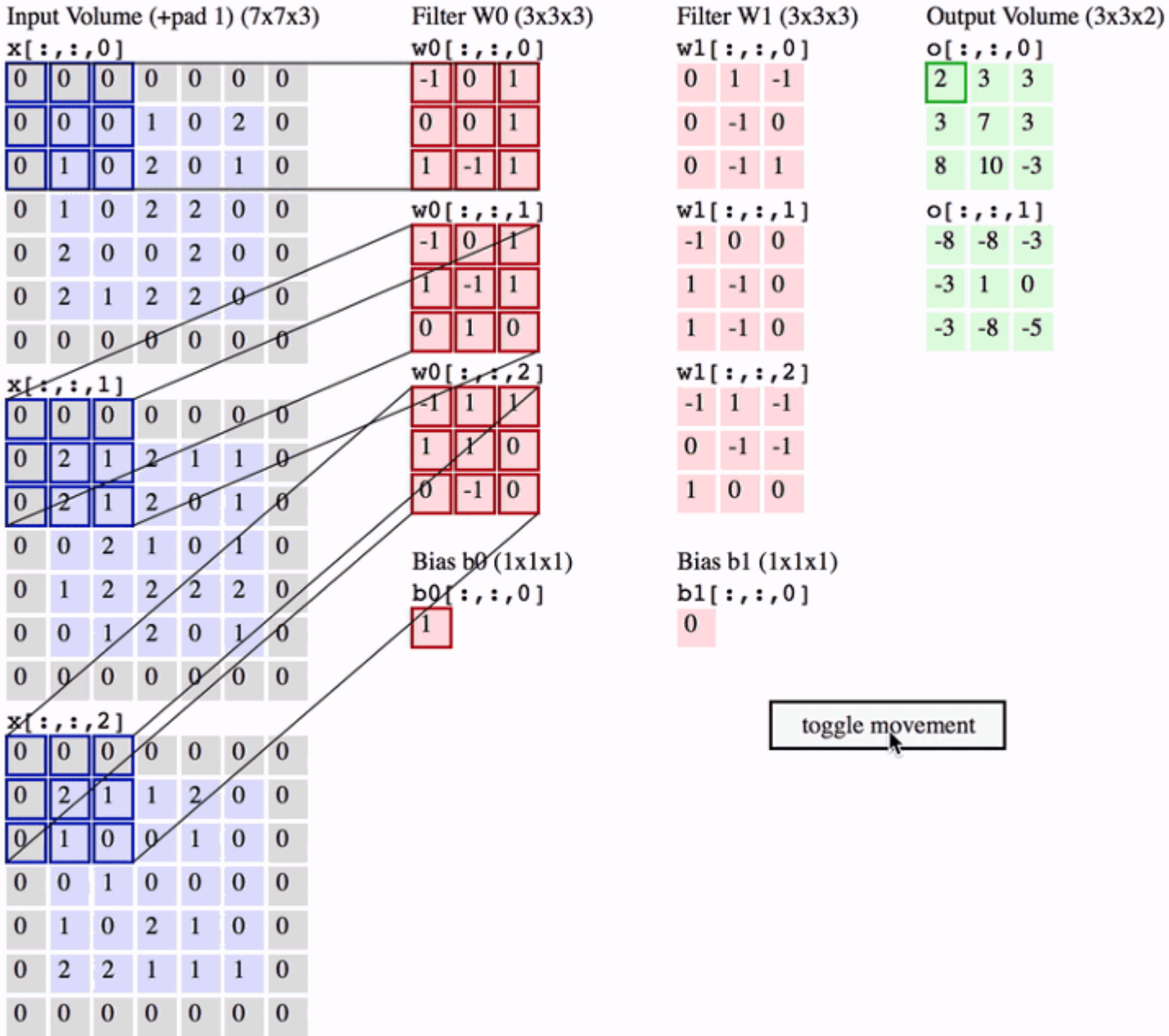
- Instead of using just a single filter, we can use multiple filters as well. **For example:** the first filter will detect vertical edges and the second filter will detect horizontal edges from the image.
- **The output will be 4 X 4 X 2 output (if we have used 2 filters):**
- **Input:** n X n X n_c
- **Filter:** f X f X n_c
- **Padding:** p
- **Stride:** s
- **Output:** [(n+2p-f)/s+1] X [(n+2p-f)/s+1] X n_c'



n_c is the number of channels in the input and filter,

while n_c' is the number of filters.

Example:


Input Volume (+pad 1) (7x7x3)
x[:,:,0]
0 0 0 0 0 0 0
0 0 0 1 0 2 0
0 1 0 2 0 1 0
0 1 0 2 2 0 0
0 2 0 0 2 0 0
0 2 1 2 2 0 0
0 0 0 0 0 0 0
x[:,:,1]
0 0 0 0 0 0 0
0 2 1 2 1 1 0
0 2 1 2 0 1 0
0 0 2 1 0 1 0
0 1 2 2 2 2 0
0 0 1 2 0 1 0
0 0 0 0 0 0 0
x[:,:,2]
0 0 0 0 0 0 0
0 2 1 1 2 0 0
0 1 0 0 1 0 0
0 0 1 0 0 0 0
0 1 0 2 1 0 0
0 2 2 1 1 1 0
0 0 0 0 0 0 0
Filter W0 (3x3x3)
w0[:,:,0]
-1 0 1
0 0 1
1 -1 1
w0[:,:,1]
-1 0 1
1 -1 1
0 1 0
w0[:,:,2]
-1 1 1
1 1 0
0 -1 0
Bias b0 (1x1x1)
b0[:,:,0]
1
Filter W1 (3x3x3)
w1[:,:,0]
0 1 -1
0 -1 0
0 -1 1
w1[:,:,1]
-1 0 0
1 -1 0
1 -1 0
w1[:,:,2]
-1 1 -1
0 -1 -1
1 0 0
Bias b1 (1x1x1)
b1[:,:,0]
0
Output Volume (3x3x2)
o[:,:,0]
2 3 3
3 7 3
8 10 -3
o[:,:,1]
-8 -8 -3
-3 1 0
-3 -8 -5
toggle movement

Next Part

- Pooling and Fully connected